Wstęp do programowania- wykład 4

1. Łączenie operatorów porównania



👉 Porównania można łączyć w jednym warunku, zamiast używać dwóch warunków połączonych przez and:

```
if 90 <= x <= 110:
print('Około 100')
```

Metoda .get() słownika

👉 Pobiera wartość przypisaną do klucza. Jeśli klucza nie ma, zwraca podaną wartość domyślną; jeśli jej nie podamy — zwraca None.

```
slownik = {'kot': 'cat', 'pies': 'dog'}

print(slownik.get('kot', '???'))          cat          👉Wynik jest napisany kolorem fioletowym
print(slownik.get('ryba', '???'))   # ??? print(slownik.get('ryba'))    None
print(slownik.get('ryba', 'ryba'))   ryba
```

👉 Lukier składniowy – wygodniejszy zapis kodu, który nie zmienia jego podstawowego działania.

2. Krotki (tuple)

👉 Krotka przechowuje kilka elementów i jest niezmienna — po utworzeniu nie można przypisywać nowych wartości do jej elementów. Indeksy zaczynają się od 0.

```
wspolrzedne = (4, 3)
student = ('Jan', 'Nowak', 'bioinformatyka', 123456)
print(wspolrzedne[0])          4

print(student[1])          Nowak

student[2] = 'biologia'          TypeError
```

👉Można przypisać elementy krotki do kilku zmiennych jednocześnie:

imie, nazwisko, kierunek, numer = student

👉 Krotki porównuje się element po elemencie, zaczynając od pierwszego. Krotka może być kluczem słownika, jeśli wszystkie jej elementy są hashowalne.

3. Złożone operatory przypisania

👉 Skrócony zapis działań na zmiennych:

Zapis	Oznacza
x += 5	x = x + 5
x -= 2	x = x - 2
x *= 3	x = x * 3

Zapis	Oznacza
x /= 2	x = x / 2
x %= 2	x = x % 2
x **= 2	x = x ** 2

Operator % zwraca resztę z dzielenia.

print(10 % 3) 1

print(12 % 4) 0

4. Funkcje wbudowane

range() – generowanie liczb

Składnia: range(stop), range(start, stop) lub range(start, stop, krok). Wartość stop nie jest uwzględniana.

list(range(5)) # [0, 1, 2, 3, 4]

list(range(3, 7)) [3, 4, 5, 6]

```
list(range(1, 10, 2))          [1, 3, 5, 7, 9]
list(range(5, 0, -1))          [5, 4, 3, 2, 1]
```

enumerate() – indeks i element

```
imiona = ['Anna', 'Bartek', 'Celina']

for i, imie in enumerate(imiona): print(i, imie)
```

0 Anna, 1 Bartek, 2 Celina.

sorted() – sortowanie

👉 Zwraca nową, posortowaną listę i nie zmienia oryginalnej kolekcji.

```
liczby = [3, 2, 6, 4]

print(sorted(liczby))                [2, 3, 4, 6]
print(sorted(liczby, reverse=True))  [6, 4, 3, 2]
```

sum() – sumowanie

```
sum([2, 3, 5])                 10
```

5. Domyślne argumenty funkcji

👉 Argument domyślny przyjmuje ustaloną wartość, jeśli nie podamy go podczas wywołania. Argumenty domyślne umieszczamy po wymaganych.

```
def powitaj(imie, wiadomość='Witaj'):
        print(f'{wiadomość}, {imie}!')
powitaj('Jan')                 Witaj, Jan!
powitaj('Anna', 'Cześć')       Cześć, Anna!
```

Funkcja print() i argument end

👉 Domyślnie print() kończy wypisywanie znakiem nowej linii \n. Argument end pozwala zmienić zakończenie.

```
print('Witaj,', end=' ')
print('Świecie!')
```

Witaj, Świecie!

6. Zbiory (set)

👉 Zbiór przechowuje unikalne elementy — nie zawiera powtórzeń, nie ma indeksów, a kolejność elementów nie jest gwarantowana.

```
liczby = {1, 2, 3, 3}
print(liczby) # {1, 2, 3}
pusty_zbior = set()                    {} oznacza pusty słownik
```

Sprawdzanie przynależności

```
if 2 in liczby:
    print('Należy do zbioru')
if 5 not in liczby:
    print('Nie należy do zbioru')
```

Działania na zbiorach

Dla zbiorów a i b:

- 👉 a | b – suma: elementy należące do co najmniej jednego zbioru.
- 👉 a & b – część wspólna: elementy należące do obu zbiorów.
- 👉a - b – różnica: elementy należące do a, ale nie do b.

```
a = {1, 2, 3, 4}
b = {2, 3, 5}
```

```
print(a | b)                    {1, 2, 3, 4, 5}
print(a & b)                    {2, 3}
print(a - b)                    {1, 4}
```

👉Zbiory pomagają usuwać powtórzenia i sprawdzać przynależność elementów.

7. Listy, zbiory i słowniki składane

👉 Pozwalają tworzyć kolekcje w krótszy sposób.

Lista składana

```
sześciany = [i**3 for i in range(5)]
[0, 1, 8, 27, 64]

wyrazy = ['jabłko', 'kiwi', 'banan', 'arbuz', 'ananas']
dlugie = [w for w in wyrazy if len(w) > 5]
['jabłko', 'ananas']
```

Zbiór składany

👉 Używa klamer {} i usuwa powtórzenia.

```
kwadraty = {i**2 for i in range(5)}
{0, 1, 4, 9, 16}
```

Słownik składany

```
wyrazy = ['kot', 'pies', 'ryba']
dlugosci = {w: len(w) for w in wyrazy}
{'kot': 3, 'pies': 4, 'ryba': 4}
```

👉 Odwracanie słownika polsko-czeskiego:

```
slownik_cz_pl = {
cz: pl for pl, cz in slownik_pl_cz.items()
}
```

!Uwaga: jeśli różne klucze mają tę samą wartość, podczas odwracania słownika wpisy mogą zostać nadpisane.

8. Wartości prawdziwe i nieprawdziwe

👉W warunkach Python traktuje obiekty jako prawdziwe lub nieprawdziwe.

👉Nieprawdziwe wartości: False, 0, 0.0, pusty napis '', pusta lista [], pusta krotka (), pusty słownik {}, pusty zbiór set() oraz None.

Typowe wartości prawdziwe to np. True, 2.5, 'szynka' i [1, 2].

```
if lista:
        print('Lista nie jest pusta')
else:
        print('Lista jest pusta')
```

👉 None oznacza brak wartości.

9. Importowanie modułów

👉 Moduł to plik Pythona zawierający kod, np. funkcje. Aby z niego korzystać, należy go zaimportować.

```
import math
print(math.sqrt(9))
import statistics as stat
alias
```

```
from math
import sqrt print(sqrt(9))
```

👉 Nie zaleca się from math import *, ponieważ utrudnia to ustalenie, skąd pochodzą używane nazwy.

👉 Biblioteka standardowa zawiera moduły dostarczane razem z Pythonem. Biblioteki zewnętrzne instaluje się osobno.

10. Najważniejsze moduły

math – matematyka

```
import math
math.pi                    liczba pi
math.sqrt(9)               3.0 – pierwiastek
math.log(8, 2)             3.0 – logarytm o podstawie 2
 math.sin(x)               sinus
math.cos(x)                cosinus
math.tan(x)                tangens
```

👉 Funkcje trygonometryczne przyjmują argumenty w radianach, a nie w stopniach.

statistics – statystyka

```
import statistics
statistics.mean([2, 5, 4, 4])          3.75 – średnia
statistics.median([1, 2, 4, 8, 3])     3 – mediana
```

👉 Średnią można też obliczyć jako sum(lista) / len(lista) dla niepustej listy liczb.

random – losowanie

```
import random
random.randint(1, 6)                       liczba całkowita od 1 do 6 włącznie
random.choice(['jabłko', 'gruszka', 'banan'])   losowy element
```

👉 randint(a, b) uwzględnia obie granice. choice() wybiera jeden element z niepustej kolekcji.

urllib.request – pobieranie danych z Internetu

```
import urllib.request
with urllib.request.urlopen('https://www.example.com') as response:
        dane = response.read().decode('utf-8')
```

```
print(dane)
```

👉 urlopen() otwiera zasób internetowy, read() odczytuje dane, a decode('utf-8') zamienia bajty na tekst.